

INDEPENDENT_ANALYSIS_6e6b030

9

LLMsVerifier seed-at-boot — safety analysis (READ-ONLY, no changes made)

Date: 2026-06-16 Analyst scope: helix_translate parent + llms_verifier/ submodule. Method: code inspection only. No edits, no commits, no DB/config mutation, nezha untouched. Anti-bluff: every claim below is quoted from source with file:line; no guessing (§11.4.6).

VERDICT (up front)

Enabling the LLMsVerifier wiring tonight is NOT safe and MUST remain BLOCKED.

Two independent reasons, both proven from code:

1. **The seed produces an EMPTY DB.** The deployed verifier's seed source is llms_verifier/llm-verifier/config.yaml, whose llms: list is literally empty: llms: [] (config.yaml:16). seed.FromConfig iterates cfg.LLMs (seed.go:60 for i := range cfg.LLMs); an empty slice = zero iterations = zero rows. The commit message itself says "no-op when llms: empty". So the new seed code does NOT change the deployed reality: /api/models stays count:0. The §11.4 forensic concern from rev88 (empty verifier degrades translation) is unchanged.
2. **An empty verifier in HTTP-bridge mode HARD-BREAKS translation — there is NO graceful fallback to the working direct DeepSeek/Groq path.** If the translation bridge is pointed at the (empty) deployed verifier via LLMsverifier_API_URL, BestTranslator/BestModel return "no strongest verified model" and createTranslator propagates that as an HTTP error — every /api/v1/translate* request fails.

Precondition for non-degrading enablement: the verifier DB must contain ≥ 1 model that passes the helixtranslate-side score/verified gate AND be resolvable to a real provider key — which requires populating config.yaml's llms: (or a real verification run), NOT merely deploying the seed-at-boot code.

Q1 — Does commit 6e6b0309 actually seed config.yaml llms: into the DB at boot? Is it tested?

YES, the code genuinely seeds — for non-empty llms: — and it IS tested.

```
git -C llms_verifier show 6e6b0309 --stat:
```

```
llm-verifier/cmd/main.go      | 11 +++
llm-verifier/seed/seed.go     | 152 +++++
llm-verifier/seed/seed_test.go | 163 +++++
```

HEAD of submodule = 6e6b030986dd... and the parent gitlink points at it (git ls-tree HEAD llms_verifier → 160000 commit 6e6b0309...), matching parent commit f6c1c93.

Boot wiring — llm-verifier/cmd/main.go runServer() (the diff, after DB init):

```
// Seed the model DB from config.yaml `llms:` at boot ...
if res, serr := seed.FromConfig(cfg, db); serr != nil {
    log.Printf("seed: non-fatal - failed to seed models from
               config: %v", serr)
} else if res != nil {
    log.Printf("seed: providers +%d (reused %d) models +%d
               (reused %d) skipped-entries %d", ...)
}
server := api.NewServer(cfg, db)
```

So runServer() DOES call the seed after DB init, before serving, and it is non-fatal (a seed error never blocks the server).

Seed logic — llm-verifier/seed/seed.go FromConfig (the load-bearing loop):

```
func FromConfig(cfg *config.Config, db dbWriter) (*Result, error)
{
    if cfg == nil { return nil, fmt.Errorf("seed: nil config") }
    if db == nil { return nil, fmt.Errorf("seed: nil database") }
    res := &Result{}
    for i := range cfg.LLMs { // ← seed.go:60: empty
        slice = 0 iterations
        llm := cfg.LLMs[i]
        if llm.Name == "" { res.Skipped = append(...); continue }
        providerID, created, err := upsertProvider(db, llm) //
        idempotent
        ...
        modelCreated, err := upsertModel(db, providerID, llm) //
        idempotent
        ...
    }
    return res, nil
}
```

upsertProvider/upsertModel are genuinely idempotent (GetProviderByName / ListModels by (provider_id, model_id) before CreateProvider/CreateModel — seed.go). cfg.LLMs is real config (config/config.go:100 LLMs []LLMConfig \mapstructure:"llms").

Tests (llm-verifier/seed/seed_test.go, 6 tests) use a REAL in-memory SQLite DB with the production schema (database.New(":memory:"), not a mock — CONST-050 compliant). Notably:

- TestFromConfig_SeedsModelsFromConfig — RED-baseline: asserts empty DB before, then 2 providers + 2 models created (deepseek + groq), and that db.ListModels (the exact call /api/models uses) returns them.
- TestFromConfig_Idempotent — two passes → 0 new rows, 1 reused.
- TestFromConfig_SkipsEmptyName, _ModelIDFallsBackToName, _NilArgs.

Conclusion Q1: the seed code is real, idempotent, and tested. The commit message's "config.yaml llms: -> DB seed at boot" is accurate **conditional on llms: being non-empty**. (Note: I did not run go test — root disk near full and read-only mandate; the test SOURCE is the evidence here.)

Q2 — Is enabling SAFE now? Would seed-at-boot still produce an empty DB if llms: is []?

Still UNSAFE. With llms: [], seed-at-boot still produces an EMPTY DB.

Decisive fact — llms_verifier/llm-verifier/config.yaml:16:

```
llms: []
```

This is the verifier's own config (its database.path: ./llm-verifier.db, api.port: '8080'). With cfg.LLMs empty, FromConfig's loop body never runs → ProvidersCreated=0, ModelsCreated=0 → DB still count:0 → /api/models still returns an empty set. The seed-at-boot commit therefore does NOT, by itself, change the rev88 HONEST-BLOCKED situation.

Precondition for the seed to be non-degrading: the verifier config.yaml llms: list must be populated with real provider+model entries (e.g. deepseek, groq, exactly as the test fixtures show) — OR a real verification pass must populate the DB — AND those entries must clear the helixtranslate-side gate (VerificationStatus=="verified" + OverallScore > MinScoreThreshold; see Q3). Seeded models are written with VerificationStatus: "pending" (seed.go upsertModel: VerificationStatus: "pending"), NOT "verified" — so even a populated llms: would NOT pass the helixtranslate listVerifiedModels filter (which requires m.VerificationStatus != "verified" { continue }, verifier_handlers.go:137) without a subsequent real verification run flipping status to "verified". This is a second, independent gap.

Q3 — On 0 models, does the helixtranslate side degrade gracefully or break translation?

There are TWO distinct surfaces. They behave differently:

(a) `/api/v1/verified-models` reporting endpoint — degrades gracefully (read-only, harmless)

`pkg/api/verifier_handlers.go` `listVerifiedModels` (registered at `verifier_handlers.go:100`) calls `h.client.GetVerifiedModels` and on a 0-model result simply returns `{"models": [], "count": 0}` (200 OK). It is gated by `requireEnabled` (503 when `cfg.LLMsVerifier.Enabled == false`). This endpoint does NOT drive translation — it is pure reporting. Harmless either way.

(b) The TRANSLATION path — `createTranslator` → `bridge` — can HARD-BREAK

This is the load-bearing path. `pkg/api/handler.go` `createTranslator` (`handler.go:887`) builds every translator via the bridge:

```
return h.bridgeFor()(context.Background(),
    selection.TaskRequirements{
        SourceLang: sourceLang, TargetLang: targetLang,
    })
```

`bridgeFor` lazily opens a real bridge with the ZERO Options (`handler.go:87-89`): `bridge.Open(c, bridge.Options{})`.

`pkg/bridge/bridge.go` `Open` resolves the mode from env (`bridge.go:107-110`):

```
apiURL := opts.APIURL
if apiURL == "" { apiURL = getenv("LLMSVERIFIER_API_URL") }
...
if apiURL != "" {                                     // HTTP mode → points at the
    deployed verifier
    vc := verifier.NewClient(&verifier.Config{APIURL:
        apiURL, ...})
    if err := vc.Ping(ctx); err != nil { return nil,
        fmt.Errorf("...unreachable...") }
    factory.SetClient(vc); b.source = "http"; return b, nil
}
// else in-process mode (canonical 00TB): verify real provider
// APIs from env keys
```

- **HTTP mode (LLMSVERIFIER_API_URL set, pointing at the empty deployed verifier):** the factory's registry is fed from `/api/models` (0 models). `BestModel` (`bridge.go:259-275`) → `rankedModels()` empty → return `ModelInfo{}`, `fmt.Errorf("bridge: no strongest verified model: registry holds no verified models")`. `createTranslator` returns that error → `translateText/translateFB2` answer HTTP 400/500. **Translation BREAKS. There is NO fallback to the in-process / direct DeepSeek-Groq path** — `Open` returns in HTTP branch before in-process is ever reached

```
(bridge.go:139-142 return b, nil).CreateTranslatorWithFallback
confirms the hard-error: verified_factory.go:262 return
nil,nil,fmt.Errorf("no verified models available and
LLMsVerifier unreachable: %w", err).
```

- **In-process mode (no LLMSVERIFIER_API_URL — current default):** the bridge verifies the env provider keys (DEEPSEEK/GROQ/...) directly against the real provider APIs and selects the strongest. This IS the working direct path. It hard-errors only if NO provider keys are present (bridge.go:160-165), which is the intended honest no-key error, unrelated to the verifier server.

Conclusion Q3: translation does NOT auto-degrade to a safe fallback when the verifier returns 0 models in HTTP mode — it fails. The only thing that keeps the direct path working today is that LLMSVERIFIER_API_URL is unset (so the bridge runs in-process). Pointing the bridge at the empty deployed verifier would break translation.

Current wiring state (config evidence)

- config.json has **no llmsverifier section at all** (top keys: server, security, translation, preparation, distributed, logging). So cfg.LLMSVerifier.Enabled is the struct zero-value (false) unless LLMSVERIFIER_ENABLED=true is exported (internal/config/config.go:336–337).
 - config.json sets no LLMSVERIFIER_API_URL; it is read only from env (internal/config/config.go:339–341) and by the bridge from env (bridge.go:109). Absent ⇒ bridge defaults to in-process (the working path).
 - cmd/server/main.go:144–150 shows Enabled ONLY toggles the read-only verifier routes (503 vs serve); it does NOT switch the translation bridge. (Note: cmd/api-server/main.go:207 hardcodes APIURL: "http://localhost:8080" for its verifier handler, but that is the reporting handler, not the translation bridge.)
-

Q4 — Safest next step (operator away, zero-risk, direct path must NOT degrade)

Recommendation: KEEP THE WIRING BLOCKED tonight. Do nothing that sets LLMSVERIFIER_API_URL or otherwise routes the translation bridge at the empty deployed verifier. Take NO action tonight (the safest action is none).

Rationale (all evidence above):

- The bridge currently runs in-process (no LLMSVERIFIER_API_URL), so the working direct DeepSeek/Groq path is intact. Leaving env/config untouched preserves it with zero risk.
- The seed-at-boot code is correct but a NO-OP against the deployed config.yaml (llms: []), so deploying/enabling it changes nothing useful and risks breaking translation if it is mistakenly paired with pointing the bridge at the verifier.

- Enabling would only be non-degrading once BOTH preconditions hold, neither of which is met tonight:
 1. The verifier config.yaml llms: is populated with real entries (the seed then writes them as pending), AND
 2. A real verification pass flips those to VerificationStatus="verified" with OverallScore > MinScoreThreshold so the helixtranslate gate admits them.
- These are populate-then-verify-then-wire steps that change external/runtime state and (per §11.4.101) are reversible but their correctness cannot be proven from current evidence without the operator's provider keys + a live run — so they are appropriately operator-gated, not an autonomous overnight action.

If any progress is desired without risk, the ONLY safe, reversible prep is authoring (not deploying) a populated config.yaml llms: fixture + a verification-run plan for operator review — but even that should not touch the running deployment or the translation env tonight.

Key file:line references

- llms_verifier/llm-verifier/config.yaml:16 — llms: [] (empty seed source — decisive).
- llms_verifier/llm-verifier/seed/seed.go:60 — for i := range cfg.LLMs (no-op when empty); upsertModel writes VerificationStatus: "pending".
- llms_verifier/llm-verifier/cmd/main.go (commit 6e6b0309 diff) — seed.FromConfig(cfg, db) after DB init in runServer(), non-fatal.
- llms_verifier/llm-verifier/seed/seed_test.go — 6 real-SQLite tests incl. RED-baseline + idempotency + empty-name skip.
- pkg/api/handler.go:887 createTranslator → pkg/api/handler.go:83–97 bridgeFor → bridge.Open(ctx, bridge.Options{}).
- pkg/bridge/bridge.go:107–150 — env-driven HTTP-vs-in-process mode; HTTP branch returns early (no in-process fallback).
- pkg/bridge/bridge.go:271–272 BestModel empty-registry hard error.
- pkg/translator/llm/verified_factory.go:261–268 CreateTranslatorWithFallback "no verified models available" hard error.
- pkg/api/verifier_handlers.go:100,109–161 — /api/v1/verified-models read-only, returns empty list gracefully; :137 requires VerificationStatus=="verified".
- internal/config/config.go:336–341 — LLMSVERIFIER_ENABLED / LLMSVERIFIER_API_URL read from env only.
- cmd/server/main.go:144–150 — Enabled toggles only the reporting routes, not the translation bridge.
- config.json — no llmsverifier section; no LLMSVERIFIER_API_URL.